Infrastructure as Code with Terraform: Rebuild the Weather Advisor

Course: ITMO 444/544 Cloud Computing / Cloud Security
Level: Advanced
Estimated Time: 150 to 180 minutes
Budget: Under **$1.00** of AWS spend. The stack costs about the same per hour as lab06, and the Gemini free tier costs nothing.
Prerequisites: Lab06 completed and **deleted** (run its delete scripts first). You will copy the `lab06` folder to a new `lab07` folder in Part 0 and work only there. Everyone does this lab in Terraform.

---

1. Learning Objectives

By the end of this lab you will be able to:

1. Explain the difference between **imperative** infrastructure code (your bash scripts, your boto3 code) and **declarative** infrastructure code (Terraform), and say what each one is best at.
2. Install Terraform and run the whole workflow: `init`, `fmt`, `validate`, `plan`, `apply` and `destroy`.
3. Read a **plan** and tell, before anything changes, what Terraform will create, update in place, or destroy and recreate.
4. Rebuild the whole Lab06 weather advisor stack (two tiers, one ALB, two Auto Scaling Groups, DynamoDB, IAM roles, a Gemini key in Parameter Store) in about **430 lines of Terraform**, with **no delete script**.
5. Use **variables, outputs, data sources,** `for_each` **and** `lifecycle` to avoid repeating yourself and to keep a secret out of your files and out of Terraform's state.
6. See how Terraform **orders the work itself** from the references between resources, instead of you writing the order down.
7. Detect and repair **drift** (a change made by hand outside Terraform) and roll out a code change with an **instance refresh**.
8. Explain how Terraform is **like AWS Lambda** (and where the comparison stops), and say what would change in this stack if the advisor were a Lambda function.
9. Destroy everything with one command and **prove nothing that costs money is left**.

---

2. Why This Matters

In lab04 you built the stack with a bash script. In lab05 you rebuilt it in Python with boto3. Both are **imperative**: you write the steps, in order, and you decide what happens when a step is run twice, when something already exists, or when something fails halfway. That is why your scripts are full of `ensure_` helpers, `|| true`, `sleep` and retry loops, and why every create script needs a matching delete script.

Terraform is **declarative**. You write down the **end state** you want: this load balancer, these two target groups, this table, these roles. Terraform compares that with what exists, works out the steps, runs them in a safe order, and remembers what it built so it can change or remove exactly that later.

2.1. What really gets shorter (measured)

It is tempting to say "Terraform is less code." For this lab that is **not true**, and you will count it yourself in Part F. The reference Lab06 scripts are about **420 lines** of bash. The Terraform in this lab is about **430 lines**, and it includes things the bash never did (a Terraform-owned security group, an AMI lookup, an automatic instance refresh, input validation, tags on everything). The saving is in **kinds of work**, not in lines.

Work	In bash (lab06)	In Terraform (lab07)
Delete everything	A 76 line delete script with ordered steps and retry loops	**Nothing to write.** `terraform destroy`
Order of creation	You wrote it (ALB first, so the advisor's user-data has its address)	**Nothing to write.** A reference to `aws_lb.main.dns_name` creates the order
"Does it exist already?"	An `ensure_` function per resource type, or `|| true`	**Nothing to write.** Terraform compares your files with its state
Waiting for things	`wait` commands, `sleep 15` for IAM, retry loops for security groups	The provider waits and retries for you
One thing per tier, twice	A helper function called twice	`for_each` over one map
Finding the VPC, subnets and AMI	`describe-*` calls with `--query`	`data` blocks
What will this change?	You run it and find out	`terraform plan` shows the change first
Someone changed it by hand	A re-run skips anything that exists, so it never notices	`terraform plan` shows the drift
New code or setting reaches the instances	Publish a template version, then terminate the instance yourself	`instance_refresh` replaces the instance

2.2. What gets harder

- **A new thing to learn**: a language (HCL), a workflow, and a **state file** that you must protect.
- **The state can leak secrets.** Anything Terraform reads or writes can end up in `terraform.tfstate` in plain text. You will see how to keep the Gemini key out of it.
- **Some jobs are still imperative.** Storing the real Gemini key is a one-time action, so it stays a CLI command.
- **Mistakes are fast.** A single `terraform destroy` removes everything it manages. That is the point, and it is also why you read the plan.

---

3. How Terraform Is Like AWS Lambda (and Where the Comparison Stops)

These two are different kinds of thing, so be careful with the comparison. **Lambda is a service that AWS runs** and bills you for each time your code runs. **Terraform is a program you run** (on your laptop or in a pipeline) that calls AWS APIs and then exits. But they share a design idea that is worth seeing: *you hand over a small, self-contained piece, and the platform does the repetitive, error-prone work around it.*

Idea	AWS Lambda	Terraform
You supply one small unit	A function: a few dozen lines of code	A configuration: a few dozen lines that describe the end state
The platform does the plumbing	Servers, operating system patching, scaling, health checks	Call order, retries, waiting, comparing before and after
You stop managing the machinery	No instance, no Auto Scaling Group, no AMI	No `ensure_` helpers, no delete script, no order of steps
Runs on demand, then stops	Starts when an event arrives and ends when the function returns. Nothing sits idle	Starts when you run `plan` or `apply` and exits. There is no Terraform process running between runs
Same input gives the same result	A well-written function is idempotent: a retried event does not double its effect	A well-written configuration is idempotent: a second `apply` changes nothing
It acts through a role	The function runs as an IAM execution role. Least privilege applies	Terraform acts as **your** credentials (or a pipeline role). Least privilege applies to it too
The code is the record	The function code says what runs	The `.tf` files say what exists, and they can be reviewed and versioned like any code

Where the comparison stops:

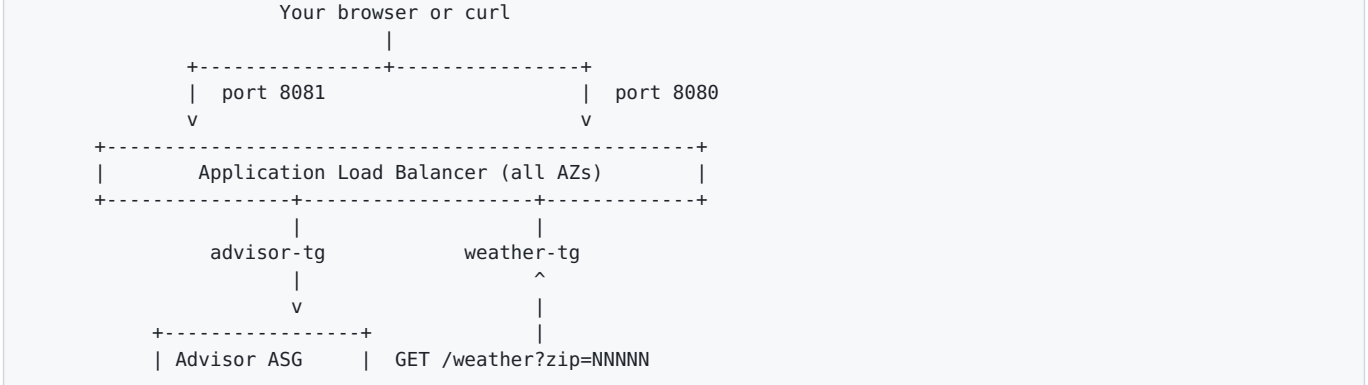
	AWS Lambda	Terraform
What it is	A **runtime** service. It runs your application code at request time	A **deployment** tool. It runs before your application exists and never handles a request
What it keeps	Nothing between invocations (stateless)	A **state file** that records what it built. Losing it is a serious problem
What it costs	You pay per invocation and per millisecond	The tool is free. The resources it creates cost money
What it can do	Run code up to a time limit, triggered by events	Create almost any cloud resource, including a Lambda function
Typical failure	A timeout, a cold start, a permission error	A plan you did not read, drift, a lost or leaked state file

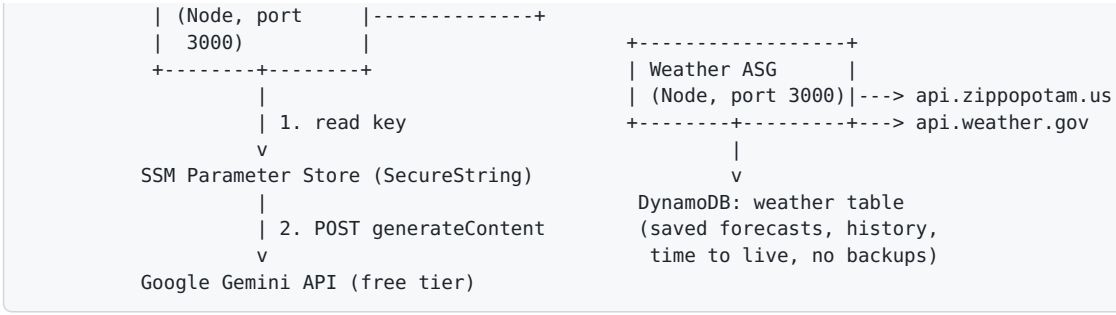
The two even meet. Terraform has a resource called `aws_lambda_function`, and **the advisor tier of this lab could be a Lambda function** behind the same load balancer. In Part F you will work out, line by line, what would disappear from your Terraform if it were. That exercise is the best way to feel the shared idea: in both cases, the less machinery you describe, the less you have to get wrong.

---

4. What You Are Building

This is the Lab06 stack, with Gemini for the advice and DynamoDB for the storage, **described in Terraform**.





What is different from lab06:

- **Terraform owns the app security group.** In lab06 the script edited a security group from lab02, revoking one rule and adding another. Here Terraform creates a new group that allows port 3000 **only from the load balancer's group**, and removes it again on destroy.
- **The advisor uses Gemini, and only the weather tier has a table.** The advisor keeps no data of its own.
- **Terraform deletes the Gemini parameter too.** Lab06 left it in place. Here the parameter is part of the stack, so you enter the key again each time you rebuild.
- **No `.env` file.** Settings live in `terraform.tfvars`. The `.env` file from earlier labs is not read by Terraform.

4.1. Cost

Item	Cost
Terraform itself	Free
Application Load Balancer	About $0.03 an hour
Two `t3.micro` instances	About $0.02 an hour
Public IPv4 addresses (the ALB's, and one per instance)	About $0.02 an hour
EBS volumes	Under $0.002 an hour
DynamoDB on demand	Fractions of a cent
Parameter Store standard parameter	Free
Gemini API free tier	Free, with rate limits

The stack costs about **$0.07 an hour** (estimates from public prices, so check the pricing pages). A 3 hour session is about 21 cents, and the stack crosses $1.00 after roughly 14 hours. **Treat 8 hours as your own hard stop.** Terraform makes teardown one command, so there is no excuse to leave it running.

4.2. What you send to a free AI service

> ⚠ Free-tier AI services are usually allowed to use your prompts to improve their products, and a human may review them. The advisor sends only a **zip code and the public forecast text**. It never sends a name, an email address, a street address or an account ID. Keep it that way, and create your Gemini key from a personal Google account, not one that holds school or work data.

---

5. Terraform in Ten Minutes

Read this once, then keep it open while you work.

Files. Terraform reads every `.tf` file in the folder as one configuration. The file names are for you. This lab splits them by topic (`network.tf`, `iam.tf`, and so on).

Blocks. Everything is a block with a type and labels:

Block	What it is	Example
`terraform`	Which Terraform and provider versions this code needs	`required_version = ">= 1.5.0"`
`provider`	How to talk to AWS (region, default tags)	`provider "aws" { region = var.aws_region }`
`resource`	A thing Terraform **creates and manages**	`resource "aws_lb" "main" { ... }`
`data`	A thing that **already exists**, only looked up	`data "aws_vpc" "default" { default = true }`
`variable`	An input you can set	`variable "key_name" { type = string }`
`locals`	A named value computed inside the code	`locals { app_port = 3000 }`
`output`	A value to print after apply	`output "advisor_url" { ... }`

Addresses and references. A resource is named `TYPE.NAME`, for example `aws_lb.main`. You use one resource's value in another with a reference such as `aws_lb.main.dns_name`. **That reference is the dependency.** Terraform builds a graph from all the references and creates things in an order that satisfies it, doing independent things at the same time.

`for_each` and `count`. `for_each` over a map creates one copy per entry and names each one with its key (`aws_lb_target_group.tier["weather"]`). `count` creates a number of copies (here zero or one, to make a rule optional).

The workflow.

Command	What it does	Changes AWS?
`terraform init`	Downloads the AWS provider into `.terraform/` and writes `.terraform.lock.hcl`	No
`terraform fmt`	Tidies the formatting of your files	No
`terraform validate`	Checks syntax and references, without calling AWS	No
`terraform plan`	Compares your files, the state, and AWS, and prints what would change	No
`terraform apply`	Shows the plan, asks `yes`, then makes the changes and updates the state	**Yes**
`terraform destroy`	Plans the removal of everything in the state, asks `yes`, then removes it	**Yes**

Reading a plan.

Symbol	Meaning
`+`	Will be **created**
`~`	Will be **updated in place**
`-`	Will be **destroyed**
`-/+`	Will be **destroyed and created again** (it says which attribute forces this)
`(known after apply)`	The value does not exist until AWS creates the thing, such as an ARN

State. After every apply Terraform writes `terraform.tfstate`, its record of what it built and the real IDs. It is how Terraform knows that `aws_lb.main` is *that* load balancer. Three rules: **never edit it by hand, never commit it to Git, and never share it casually**, because it can contain secrets. Teams keep state in a shared, locked place (for example an S3 bucket), which this lab does not need.

The lock file. `.terraform.lock.hcl` records the exact provider version you got. Commit it, so everyone uses the same one.

---

6. Part 0: Install Terraform and Set Up Your `lab07` Folder

0.1. Delete the Lab06 stack

Terraform builds its own copy of everything under different names, and two copies cost twice as much. Go into `lab06` and run its delete scripts (the stack script first, then the tables script if your lab06 has one). Skip this if you already deleted it.

0.2. Install Terraform

Download the package for your computer from the official download page (see References), or use your package manager. Then check:

```
terraform version
```

You need version **1.5.0 or newer**. Terraform finds your AWS credentials the same way the AWS CLI does, so if `aws sts get-caller-identity` works, Terraform will too:

```
aws sts get-caller-identity
```

> 💡 **A note on the license.** Terraform's license changed in 2023, and an open source fork called OpenTofu exists. The commands and the files in this lab work the same in both, so you may use either. Where this document says `terraform`, `tofu` is a drop-in replacement.

0.3. Copy the previous lab

Copy lab06:

```
cp -r lab06 lab07
cd lab07
```

Move the scripts from lab06 out of the way, because Terraform replaces them. Keep them for the comparison in Part F:

```
mkdir previous
mv create_*.sh delete_*.sh previous/ 2>/dev/null
mv create_*.py delete_*.py previous/ 2>/dev/null
rm -f stack_outputs.env table_outputs.env db_outputs.env
```

You will replace the application folders in Part B, so that everyone starts from the same code.

0.4. Your `lab07` layout

```
lab07/
  .env                    (left over from earlier labs, not read by Terraform)
  <your-key>.pem
  weather-app/            (replaced in Part B)
  advisor-app/            (replaced in Part B)
  versions.tf             (new: Terraform and provider versions)
  variables.tf            (new)
  locals.tf               (new)
  network.tf              (new)
  storage.tf              (new)
  iam.tf                  (new)
  alb.tf                  (new)
  compute.tf              (new)
  outputs.tf              (new)
  user-data.sh.tftpl      (new: the boot script, as a template)
  terraform.tfvars        (new: your own values, not committed)
  testing/
    app_smoke_test.py     (replaced)
```

```
  first-taste/            (new: Part A)
  previous/               (the lab06 scripts, for comparison)
```

✅ **Checkpoint:** `terraform version` prints 1.5 or newer, `aws sts get-caller-identity` works, and `lab07/previous/` holds your old scripts.

---

7. Part A: Your First Terraform (10 minutes)

Before you build the real stack, run the whole cycle on one tiny resource. It costs nothing while it exists. Create a folder and one file:

```
mkdir first-taste
cd first-taste
```

Create `first-taste/main.tf`:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-2"
}

resource "aws_dynamodb_table" "demo" {
  name         = "itmo-444-544-terraform-demo"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "id"

  attribute {
    name = "id"
    type = "S"
  }
}

output "table_arn" {
  value = aws_dynamodb_table.demo.arn
}
```

A1. Initialize, plan, apply

```
terraform init
terraform plan
```

Read the plan. It should say `Plan: 1 to add, 0 to change, 0 to destroy`, and it lists every attribute of the table, many of them `(known after apply)`. Now create it:

```
terraform apply
```

Type `yes`. Then confirm with the CLI that the table is real, and look at the state:

```
aws dynamodb describe-table --table-name itmo-444-544-terraform-demo --query "Table.TableStatus" --output tex
terraform state list
terraform apply          # run it again
```

The second `apply` should say `No changes`. That is **idempotence**, and you did not write a single `if`.

A2. Change it in place

Add one line inside the resource, above the `attribute` block:

```
  tags = { Purpose = "first-taste" }
```

Run `terraform plan`. The table shows a `~`, an **update in place**, and a line `+ Purpose = "first-taste"`. Apply it.

A3. Change something that cannot change

DynamoDB cannot change a table's key. In the file, change **both** the `hash_key` and the `attribute` name from `id` to `name`. Run `terraform plan` (do not apply). The table shows `-/+` and a note that `hash_key` **forces replacement**. This is the most important thing a plan tells you: it warns you before you delete data. Undo your change.

A4. Destroy it

```
terraform destroy
cd ..
```

Type `yes`. The table is gone, and so is `first-taste`'s state record of it. You did not write a delete command either.

✓ **Checkpoint:** You saw `+`, `~`, `-/+` and `No changes`, and you can explain why the `-/+` plan was the one to be careful about.

---

8. Part B: The Two Applications (Unchanged Code)

The applications are **not** infrastructure, so they do not change from the lab06 design. Replace your two app folders so that everyone has the same code:

```
rm -rf weather-app advisor-app
mkdir weather-app advisor-app testing
```

Both are small Node.js programs that write **one JSON object per log line**, which systemd keeps in the instance's journal.

B1. The weather service

- `GET /weather?zip=60616` looks up the coordinates for the zip code, asks the National Weather Service for the forecast, and returns the first period. Each upstream call has an 8 second timeout and one retry. Status codes are `400` for a bad zip, `404` for an unknown zip, and `502` when an upstream service fails.
- **DynamoDB is a cache and a history.** The service reads the item for the zip first. If it is newer than `CACHE_SECONDS`, it answers from it (`"cache": "hit"`). Otherwise it fetches the forecast, saves it, and answers (`"cache": "miss"`). `GET /history` shows the ten most recent lookups.
- If DynamoDB is unreachable or denied, the service answers without it (`"cache": "unavailable"`) and leaves the table alone for 30 seconds. A storage problem never becomes a weather outage.
- It finds its table from the `WEATHER_TABLE` setting and uses the **instance role** for credentials, so there is no password anywhere.

Create `weather-app/package.json`:

```
{
  "name": "weather-app",
  "version": "1.0.0",
```

```
  "main": "app.js",
  "dependencies": {
    "@aws-sdk/client-dynamodb": "^3.600.0",
    "@aws-sdk/lib-dynamodb": "^3.600.0",
    "express": "^4.19.2"
  }
}
```

Create `weather-app/app.js`:

```
const express = require('express');
const { DynamoDBClient } = require('@aws-sdk/client-dynamodb');
const { DynamoDBDocumentClient, GetCommand, PutCommand, ScanCommand } = require('@aws-sdk/lib-dynamodb');

const app = express();
const PORT = process.env.PORT || 3000;
const ZIP_API = process.env.ZIP_API_BASE || 'https://api.zippopotam.us';
const NWS_API = process.env.NWS_API_BASE || 'https://api.weather.gov';
// The National Weather Service asks every client to identify itself.
const USER_AGENT = process.env.NWS_USER_AGENT || '(itmo-444-544 weather lab, student@example.com)';

// With no WEATHER_TABLE the service still works, it just does not store anything.
const TABLE = process.env.WEATHER_TABLE;
const CACHE_SECONDS = Number(process.env.CACHE_SECONDS || 600);
const KEEP_SECONDS = 86400; // DynamoDB deletes saved lookups about a day after they were made
const DB_RETRY_MS = 30000;  // after a database failure, leave the database alone for 30 seconds

// DYNAMODB_ENDPOINT is for local testing only. On EC2 the SDK finds the region and the role itself.
const ddb = DynamoDBDocumentClient.from(new DynamoDBClient({
  region: process.env.AWS_REGION,
  ...(process.env.DYNAMODB_ENDPOINT ? { endpoint: process.env.DYNAMODB_ENDPOINT } : {}),
  maxAttempts: 2,
}));

const log = (event, fields = {}) =>
  console.log(JSON.stringify({ ts: new Date().toISOString(), event, ...fields }));

class UpstreamError extends Error {
  constructor(stage, status) {
    super(`${stage} failed`);
    this.stage = stage;
    this.status = status;
  }
}

// Fetch JSON with a timeout and one retry. Returns null on a 404, throws UpstreamError otherwise.
async function fetchJson(stage, url, headers = {}) {
  let status = 0;
  for (let attempt = 1; attempt <= 2; attempt++) {
    try {
      const res = await fetch(url, { headers, signal: AbortSignal.timeout(8000) });
      status = res.status;
      if (res.ok) return await res.json();
      if (res.status === 404) return null;
      if (res.status < 500 && res.status !== 429) break; // a client error will not fix itself
    } catch (err) {
      status = 0; // timeout or network failure
    }
    if (attempt < 2) await new Promise((resolve) => setTimeout(resolve, 500));
  }
  throw new UpstreamError(stage, status);
}

// ---- Database helpers ------------------------------------------------------

let dbRetryAt = 0;
```

```
// Run one database call. Any failure is logged, and the database is skipped for a short time,
// so a broken database costs each request nothing instead of a wait.
async function withDb(stage, fn) {
  if (!TABLE) return { state: 'disabled' };
  if (Date.now() < dbRetryAt) return { state: 'unavailable' };
  try {
    return { state: 'ok', value: await fn() };
  } catch (err) {
    dbRetryAt = Date.now() + DB_RETRY_MS;
    log('db_error', { stage, message: err.name + ': ' + err.message });
    return { state: 'unavailable' };
  }
}

// One item per zip code: the newest lookup. Reading it is the cache check.
const findFresh = (zip) => withDb('read', async () => {
  const r = await ddb.send(new GetCommand({ TableName: TABLE, Key: { zip } }));
  const item = r.Item;
  if (item && Date.now() / 1000 - item.createdAt < CACHE_SECONDS) return item.payload;
  return null;
});

const saveLookup = (zip, payload) => withDb('write', () => {
  const now = Math.floor(Date.now() / 1000);
  return ddb.send(new PutCommand({
    TableName: TABLE,
    Item: { zip, payload, createdAt: now, expiresAt: now + KEEP_SECONDS },
  }));
});

// ---- Routes -----------------------------------------------------------------

app.get('/', (req, res) => {
  res.json({ service: 'weather', usage: '/weather?zip=60616', history: '/history' });
});

// The ten most recent lookups. The table is tiny, so a Scan and a sort is fine for this lab.
app.get('/history', async (req, res) => {
  const result = await withDb('history', async () => {
    const items = [];
    let key;
    do {
      const r = await ddb.send(new ScanCommand({ TableName: TABLE, ExclusiveStartKey: key, Limit: 100 }));
      items.push(...r.Items);
      key = r.LastEvaluatedKey;
    } while (key && items.length < 500);
    return items
      .sort((a, b) => b.createdAt - a.createdAt)
      .slice(0, 10)
      .map((i) => ({
        zip: i.zip,
        city: i.payload.city,
        state: i.payload.state,
        short_forecast: i.payload.forecast.shortForecast,
        temperature: i.payload.forecast.temperature,
        created_at: new Date(i.createdAt * 1000).toISOString(),
      }));
  });
  if (result.state === 'disabled') return res.status(503).json({ error: 'database_disabled' });
  if (result.state === 'unavailable') return res.status(503).json({ error: 'database_unavailable' });
  return res.json({ count: result.value.length, lookups: result.value });
});

app.get('/weather', async (req, res) => {
  const zip = String(req.query.zip || '');
  if (!/^\d{5}$/.test(zip)) {
    log('weather_rejected', { reason: 'invalid_zip' });
    return res.status(400).json({ error: 'invalid_zip', message: 'Give a 5 digit US zip code.' });
  }
```

```
  const started = Date.now();
  log('weather_request', { zip });

  // 0. Has anyone asked for this zip in the last few minutes? Then answer from DynamoDB.
  const cached = await findFresh(zip);
  if (cached.state === 'ok' && cached.value) {
    log('weather_success', { zip, cache: 'hit', ms: Date.now() - started });
    return res.json({ ...cached.value, cache: 'hit' });
  }

  try {
    // 1. The weather API wants coordinates, not zip codes, so look the zip code up first.
    const lookup = await fetchJson('zip_lookup', `${ZIP_API}/us/${zip}`);
    if (!lookup || !lookup.places || lookup.places.length === 0) {
      log('weather_rejected', { reason: 'zip_not_found', zip });
      return res.status(404).json({ error: 'zip_not_found' });
    }
    const place = lookup.places[0];
    const lat = Number(place.latitude).toFixed(4);
    const lon = Number(place.longitude).toFixed(4);

    // 2. Ask NWS which forecast grid covers those coordinates.
    const headers = { 'User-Agent': USER_AGENT, Accept: 'application/geo+json' };
    const point = await fetchJson('nws_points', `${NWS_API}/points/${lat},${lon}`, headers);
    if (!point) {
      log('weather_rejected', { reason: 'no_nws_coverage', zip });
      return res.status(404).json({ error: 'no_nws_coverage' });
    }

    // 3. Fetch the forecast for that grid and keep the first period (now, or the next block).
    const forecast = await fetchJson('nws_forecast', point.properties.forecast, headers);
    const period = forecast.properties.periods[0];
    const precip = period.probabilityOfPrecipitation ? period.probabilityOfPrecipitation.value : null;

    const body = {
      zip,
      city: place['place name'],
      state: place['state abbreviation'],
      latitude: lat,
      longitude: lon,
      forecast: {
        name: period.name,
        temperature: period.temperature,
        temperatureUnit: period.temperatureUnit,
        shortForecast: period.shortForecast,
        windSpeed: period.windSpeed,
        precipitationChance: precip,
        isDaytime: period.isDaytime,
      },
    };

    // 4. Keep a copy. This is both the cache for the next caller and the history.
    const saved = await saveLookup(zip, body);
    const cache = saved.state === 'ok' ? 'miss' : saved.state; // 'miss', 'disabled' or 'unavailable'
    log('weather_success', { zip, cache, ms: Date.now() - started });
    return res.json({ ...body, cache });
  } catch (err) {
    if (err instanceof UpstreamError) {
      log('weather_error', { zip, stage: err.stage, status: err.status });
      return res.status(502).json({ error: 'upstream_error', stage: err.stage });
    }
    log('weather_error', { zip, stage: 'internal', message: err.message });
    return res.status(500).json({ error: 'internal_error' });
  }
});

app.listen(PORT, () => log('weather_started', { port: PORT, table: TABLE || null }));
```

B2. The advisor service

- `GET /advice?zip=60616` calls the weather service at `WEATHER_URL` (the load balancer, port 8080), then asks Gemini for a two-sentence clothing suggestion. `GET /` serves a small page with a form.
- It reads the Gemini key from Parameter Store with its instance role and keeps it in memory for `KEY_CACHE_SECONDS`.
- If Gemini fails for any reason (placeholder key, rate limit, timeout), it logs a `gemini_error` and answers with a rule-based suggestion marked `"source": "fallback"`.
- Everything printed on the page is HTML-escaped, because the model's text is **untrusted input**.
- The prompt contains only the zip code, city, state and forecast fields (Section 4.2).

Create `advisor-app/package.json`:

```
{
  "name": "advisor-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.19.2",
    "@aws-sdk/client-ssm": "^3.600.0"
  }
}
```

Create `advisor-app/app.js`:

```
const express = require('express');
const { SSMClient, GetParameterCommand } = require('@aws-sdk/client-ssm');

const app = express();
const PORT = process.env.PORT || 3000;
const WEATHER_URL = process.env.WEATHER_URL; // for example http://<alb-dns>:8080
const GEMINI_API_BASE = process.env.GEMINI_API_BASE || 'https://generativelanguage.googleapis.com';
const GEMINI_MODEL = process.env.GEMINI_MODEL || 'gemini-3.1-flash-lite';
const KEY_PARAM = process.env.GEMINI_KEY_PARAM || '/itmo-444-544/gemini-api-key';
const KEY_CACHE_MS = Number(process.env.KEY_CACHE_SECONDS || 300) * 1000;

const ssm = new SSMClient({ region: process.env.AWS_REGION });
let cachedKey = { value: null, at: 0 };

const log = (event, fields = {}) =>
  console.log(JSON.stringify({ ts: new Date().toISOString(), event, ...fields }));

// The API key lives in SSM Parameter Store, never in user-data or an environment file.
// GEMINI_API_KEY is for running this app on your own laptop only.
async function getApiKey() {
  if (process.env.GEMINI_API_KEY) return process.env.GEMINI_API_KEY;
  if (cachedKey.value && Date.now() - cachedKey.at < KEY_CACHE_MS) return cachedKey.value;
  const out = await ssm.send(new GetParameterCommand({ Name: KEY_PARAM, WithDecryption: true }));
  cachedKey = { value: out.Parameter.Value, at: Date.now() };
  return cachedKey.value;
}

const SYSTEM_PROMPT =
  'You give a short clothing suggestion for the weather you are given. ' +
  'Reply in at most two plain sentences, with no markdown and no lists. Use only the data provided.';

// Only the zip code and the forecast go to the AI service. Nothing about the user does.
function buildPrompt(w) {
  const f = w.forecast;
  return [
    `Zip code: ${w.zip} (${w.city}, ${w.state})`,
    `Period: ${f.name}`,
    `Temperature: ${f.temperature} ${f.temperatureUnit}`,
    `Conditions: ${f.shortForecast}`,
    `Wind: ${f.windSpeed}`,
    `Chance of precipitation: ${f.precipitationChance === null ? 'unknown' : f.precipitationChance + '
```

```
percent'}`,
          ].join('\n');
        }

        async function askGemini(prompt) {
          const key = await getApiKey();
          const res = await fetch(`${GEMINI_API_BASE}/v1beta/models/${GEMINI_MODEL}:generateContent`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json', 'x-goog-api-key': key },
            body: JSON.stringify({
              systemInstruction: { parts: [{ text: SYSTEM_PROMPT }] },
              contents: [{ parts: [{ text: prompt }] }],
              generationConfig: { maxOutputTokens: 300 },
            }),
            signal: AbortSignal.timeout(15000),
          });
          if (!res.ok) {
            const err = new Error('gemini_http_error');
            err.status = res.status;
            throw err;
          }
          const data = await res.json();
          const parts = (data.candidates && data.candidates[0] && data.candidates[0].content.parts) || [];
          const text = parts.map((p) => p.text || '').join('').trim();
          if (!text) {
            const err = new Error('gemini_empty_response');
            err.status = 200;
            throw err;
          }
          return text;
        }

        // Used when the AI service is unavailable, rate limited, or the key is wrong.
        function fallbackAdvice(f) {
          const t = f.temperature;
          let base;
          if (t <= 32) base = 'Wear a heavy coat, a hat and gloves.';
          else if (t <= 50) base = 'Wear a warm jacket and long pants.';
          else if (t <= 68) base = 'Wear a light jacket or a sweater over long sleeves.';
          else if (t <= 80) base = 'A t-shirt and light pants or shorts will be fine.';
          else base = 'Wear light, breathable clothes and bring water and sunscreen.';
          const wet = (f.precipitationChance || 0) >= 40 || /rain|shower|storm|snow/i.test(f.shortForecast);
          return wet ? `${base} Bring an umbrella or a waterproof jacket.` : base;
        }

        const escapeHtml = (s) =>
          String(s).replace(/[&<>"']/g, (c) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;'
[c]));

        // Shared by the web page and the JSON endpoint. Returns { status, body }.
        async function getAdvice(zip) {
          if (!/^\d{5}$/.test(zip)) {
            log('advice_rejected', { reason: 'invalid_zip' });
            return { status: 400, body: { error: 'invalid_zip', message: 'Give a 5 digit US zip code.' } };
          }
          const started = Date.now();
          log('advice_request', { zip });

          let weather;
          try {
            const res = await fetch(`${WEATHER_URL}/weather?zip=${zip}`, { signal: AbortSignal.timeout(20000) });
            if (res.status === 400 || res.status === 404) {
              log('advice_rejected', { reason: 'weather_' + res.status, zip });
              return { status: res.status, body: await res.json() };
            }
            if (!res.ok) throw new Error('weather_status_' + res.status);
            weather = await res.json();
          } catch (err) {
            log('weather_call_failed', { zip, message: err.message });
```

```
            return { status: 502, body: { error: 'weather_service_unavailable' } };
          }

          let advice, source;
          try {
            advice = await askGemini(buildPrompt(weather));
            source = 'gemini';
          } catch (err) {
            log('gemini_error', { zip, status: err.status || 0, reason: err.name === 'TimeoutError' ? 'timeout' :
err.message });
            advice = fallbackAdvice(weather.forecast);
            source = 'fallback';
          }

          log('advice_success', { zip, source, ms: Date.now() - started });
          return { status: 200, body: { ...weather, advice, source, model: source === 'gemini' ? GEMINI_MODEL : null
        }

        app.get('/advice', async (req, res) => {
          const { status, body } = await getAdvice(String(req.query.zip || ''));
          res.status(status).json(body);
        });

        app.get('/', async (req, res) => {
          const form = `<form method="GET" action="/">
              <label>US zip code <input name="zip" pattern="[0-9]{5}" maxlength="5" required></label>
              <button type="submit">What should I wear?</button>
            </form>`;
          let result = '';
          if (req.query.zip) {
            const { status, body } = await getAdvice(String(req.query.zip));
            if (status === 200) {
              const f = body.forecast;
              result = `<h2>${escapeHtml(body.city)}, ${escapeHtml(body.state)} ${escapeHtml(body.zip)}</h2>
                <p>${escapeHtml(f.name)}: ${escapeHtml(f.temperature)} ${escapeHtml(f.temperatureUnit)},
${escapeHtml(f.shortForecast)}, wind ${escapeHtml(f.windSpeed)}</p>
                <p><strong>${escapeHtml(body.advice)}</strong></p>
                <p><small>Suggestion source: ${escapeHtml(body.source)}</small></p>`;
            } else {
              result = `<p>Sorry: ${escapeHtml(body.error)}</p>`;
            }
          }
          res.send(`<h1>Weather Advisor</h1>${form}${result}`);
        });

        app.listen(PORT, () => log('advisor_started', { port: PORT, model: GEMINI_MODEL }));
```

✅ **Checkpoint:** `ls weather-app advisor-app` shows `app.js` and `package.json` in each.

---

9. Part C: The Terraform Files

Create each file below in `lab07/`. For each one, the note says **which part of your lab06 bash it replaces**. Read them in this order, because it follows how Terraform thinks: versions, inputs, shared values, then the resources.

C1. `versions.tf`: which Terraform and which provider

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
```

```
  }
}

provider "aws" {
  region = var.aws_region

  # Every resource Terraform creates gets this tag, so you can find (and prove you removed) all of them.
  default_tags {
    tags = {
      Project = "itmo-444-544-lab07"
    }
  }
}
```

- `required_version` and `version = "~> 5.0"` pin the tools. `~> 5.0` means any 5.x version but not 6. Without a pin, a new provider version could change behavior under you.
- `default_tags` puts the tag `Project` on **every** resource Terraform creates. Later you will use that tag to prove that nothing is left.
- This replaces `export AWS_DEFAULT_REGION=...` and the repeated `--region` choices in your scripts.

C2. `variables.tf`: the inputs

```
variable "aws_region" {
  description = "Region for everything in this lab."
  type        = string
  default     = "us-east-2"
}

variable "key_name" {
  description = "Name of the EC2 key pair you created in lab02."
  type        = string
}

variable "ssh_cidr" {
  description = "Your public address as x.x.x.x/32, to allow SSH to the instances. Leave empty to allow no SSH."
  type        = string
  default     = ""

  validation {
    condition     = var.ssh_cidr == "" || can(cidrhost(var.ssh_cidr, 0))
    error_message = "ssh_cidr must be empty or a CIDR block such as 203.0.113.7/32."
  }
}

variable "contact_email" {
  description = "Goes into the User-Agent header that the National Weather Service asks every client to send."
  type        = string
}

variable "instance_type" {
  description = "Instance type for both tiers."
  type        = string
  default     = "t3.micro"
}

variable "name_prefix" {
  description = "Start of every resource name. Change it to run a second copy of the stack."
  type        = string
  default     = "itmo-444-544-lab07"
}

variable "cache_seconds" {
  description = "How long the weather service reuses a saved forecast."
  type        = number
  default     = 600
}
```

```
variable "gemini_model" {
  description = "Gemini model name. Model names change, so this is a setting."
  type        = string
  default     = "gemini-3.1-flash-lite"
}

variable "key_cache_seconds" {
  description = "How long the advisor keeps the Gemini key in memory before reading it again."
  type        = number
  default     = 60
}
```

- A variable with **no default** (`key_name`, `contact_email`) must be given a value, so Terraform asks for it or reads it from `terraform.tfvars`.
- The `validation` block on `ssh_cidr` rejects a typo **before** anything is created. In bash you would find out when the AWS call failed.
- This replaces your `.env` file.

C3. `locals.tf`: one map for both tiers

```
locals {
  # One entry per tier. Everything that exists once per tier is created from this map with for_each,
  # so adding a third tier means adding a third entry here.
  tiers = {
    weather = { listener_port = 8080 }
    advisor = { listener_port = 8081 }
  }

  app_port = 3000
}
```

This small map is the heart of the design. Every resource that exists once per tier uses `for_each = local.tiers`. It replaces the `ensure_role_and_profile`, `ensure_target_group`, `ensure_listener`, `ensure_launch_template` and `ensure_asg` helper functions you wrote, each called twice.

C4. `network.tf`: lookups and security groups

```
# ---- Things that already exist: Terraform only looks them up ----------------------------

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

# The newest Amazon Linux 2023 image, so you never copy an AMI ID again.
data "aws_ami" "al2023" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-2023.*-x86_64"]
  }
}

# ---- Security groups ---------------------------------------------------------------------
```

```
resource "aws_security_group" "alb" {
  name        = "${var.name_prefix}-alb-sg"
  description = "The load balancer: open to the internet on the two listener ports"
  vpc_id      = data.aws_vpc.default.id
}

resource "aws_security_group" "app" {
  name        = "${var.name_prefix}-app-sg"
  description = "The app instances: reachable only from the load balancer"
  vpc_id      = data.aws_vpc.default.id
}

resource "aws_vpc_security_group_ingress_rule" "alb_from_internet" {
  for_each = local.tiers

  security_group_id = aws_security_group.alb.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "tcp"
  from_port         = each.value.listener_port
  to_port           = each.value.listener_port
}

resource "aws_vpc_security_group_egress_rule" "alb_to_app" {
  security_group_id            = aws_security_group.alb.id
  referenced_security_group_id = aws_security_group.app.id
  ip_protocol                  = "tcp"
  from_port                    = local.app_port
  to_port                      = local.app_port
}

# The only way in to port 3000 is from the load balancer's security group.
resource "aws_vpc_security_group_ingress_rule" "app_from_alb" {
  security_group_id            = aws_security_group.app.id
  referenced_security_group_id = aws_security_group.alb.id
  ip_protocol                  = "tcp"
  from_port                    = local.app_port
  to_port                      = local.app_port
}

resource "aws_vpc_security_group_ingress_rule" "app_ssh" {
  count = var.ssh_cidr == "" ? 0 : 1

  security_group_id = aws_security_group.app.id
  cidr_ipv4         = var.ssh_cidr
  ip_protocol       = "tcp"
  from_port         = 22
  to_port           = 22
}

# A security group created by Terraform has no outbound rule unless you write one.
# The instances need the internet to install Node.js and to call the weather and Gemini APIs.
resource "aws_vpc_security_group_egress_rule" "app_out" {
  security_group_id = aws_security_group.app.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "-1"
}
```

- The three `data` blocks **look up** the default VPC, its subnets and the newest Amazon Linux 2023 image. They replace the `describe-vpcs`, `describe-subnets` and `AMI_ID` lines, and `AZ_A` and `AZ_B`: the load balancer and the Auto Scaling Groups simply use every default subnet.
- Terraform **creates** the two security groups. The app group's only rule for port 3000 names the **load balancer's group** as its source, so the instances cannot be reached directly. Because Terraform owns the group, there is no `revoke` and `authorize` juggling, and `destroy` removes it.
- A security group that Terraform creates has **no outbound rule** unless you write one, so `app_out` is needed. Without it the instances could not install Node.js or call the weather and Gemini APIs. This is a common first-time surprise.

- `count = var.ssh_cidr == "" ? 0 : 1` makes the SSH rule optional.

C5. `storage.tf`: the table and the key's slot

```
# The weather tier's cache and history. On-demand billing, items expire by themselves, and no backups.
resource "aws_dynamodb_table" "weather" {
  name         = "${var.name_prefix}-weather-lookups"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "zip"

  attribute {
    name = "zip"
    type = "S"
  }

  ttl {
    attribute_name = "expiresAt"
    enabled        = true
  }

  point_in_time_recovery {
    enabled = false
  }
}

# The Gemini key lives in Parameter Store, never in a file. Terraform creates the empty slot, and
# you put the real key into it with the AWS CLI. ignore_changes stops Terraform from ever looking
# at or overwriting the value, which also keeps the key out of the Terraform state file.
resource "aws_ssm_parameter" "gemini" {
  name  = "/${var.name_prefix}/gemini-api-key"
  type  = "SecureString"
  value = "not-set-yet"

  lifecycle {
    ignore_changes = [value]
  }
}
```

- The table matches lab06: on-demand billing, items expire through `expiresAt`, and point-in-time recovery is off, so there are no backups.
- **The secret handling is the important part.** Terraform creates the parameter with a placeholder value, and `ignore_changes = [value]` tells it to **never look at or overwrite the value again**. You put the real key in with the AWS CLI. The real key therefore never appears in your files **or in the state file**. Without `ignore_changes`, the next `apply` would put the placeholder back and wipe your key.

C6. `iam.tf`: roles and least privilege

```
data "aws_iam_policy_document" "ec2_trust" {
  statement {
    actions = ["sts:AssumeRole"]

    principals {
      type        = "Service"
      identifiers = ["ec2.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "tier" {
  for_each = local.tiers

  name               = "${var.name_prefix}-${each.key}-role"
  assume_role_policy = data.aws_iam_policy_document.ec2_trust.json
}
```

```
resource "aws_iam_instance_profile" "tier" {
  for_each = local.tiers

  name = "${var.name_prefix}-${each.key}-profile"
  role = aws_iam_role.tier[each.key].name
}

# The weather tier may use its own table and nothing else.
data "aws_iam_policy_document" "weather" {
  statement {
    actions   = ["dynamodb:GetItem", "dynamodb:PutItem", "dynamodb:Scan"]
    resources = [aws_dynamodb_table.weather.arn]
  }
}

# The advisor tier may read one secret and nothing else.
data "aws_iam_policy_document" "advisor" {
  statement {
    actions   = ["ssm:GetParameter"]
    resources = [aws_ssm_parameter.gemini.arn]
  }
}

resource "aws_iam_role_policy" "weather" {
  name   = "${var.name_prefix}-weather-policy"
  role   = aws_iam_role.tier["weather"].id
  policy = data.aws_iam_policy_document.weather.json
}

resource "aws_iam_role_policy" "advisor" {
  name   = "${var.name_prefix}-advisor-policy"
  role   = aws_iam_role.tier["advisor"].id
  policy = data.aws_iam_policy_document.advisor.json
}
```

- `data "aws_iam_policy_document"` lets you write a policy in Terraform syntax instead of a JSON text block in a heredoc. It replaces `trust-policy.json`, `weather-policy.json` and `advisor-policy.json` and the `rm -f` that cleaned them up.
- Each policy names the **one resource** its tier needs. The advisor policy refers to `aws_ssm_parameter.gemini.arn`, so the ARN is always the right one. In bash you built that ARN by hand from the account ID, and a typo caused an `AccessDenied` that was hard to see.
- The `for_each` roles and profiles are created in a loop, and you use them as `aws_iam_role.tier["weather"]`.

C7. `alb.tf`: the load balancer, target groups and listeners

```
resource "aws_lb" "main" {
  name               = "${var.name_prefix}-alb"
  load_balancer_type = "application"
  internal           = false
  security_groups    = [aws_security_group.alb.id]
  subnets            = data.aws_subnets.default.ids
}

resource "aws_lb_target_group" "tier" {
  for_each = local.tiers

  name        = "${var.name_prefix}-${each.key}-tg"
  port        = local.app_port
  protocol    = "HTTP"
  vpc_id      = data.aws_vpc.default.id
  target_type = "instance"

  health_check {
    path                = "/"
```

```
    matcher             = "200"
    interval            = 10
    timeout             = 5
    healthy_threshold   = 2
    unhealthy_threshold = 2
  }
}

resource "aws_lb_listener" "tier" {
  for_each = local.tiers

  load_balancer_arn = aws_lb.main.arn
  port              = each.value.listener_port
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.tier[each.key].arn
  }
}
```

Two `for_each` resources, a few attributes each. In lab06 this was a `ensure_target_group` function, a `ensure_listener` function, and a block that created the ALB and waited for it to become active. Terraform waits for the ALB for you.

C8. `user-data.sh.tftpl` and `compute.tf`: the instances

First the boot script as a **template**. The `${...}` parts are filled in by Terraform, and the `%{ for ... }` loop writes one `Environment=` line per setting:

```
        #!/bin/bash
        dnf update -y
        dnf install -y nodejs
        mkdir -p /opt/app
        echo "${app_js_b64}" | base64 -d > /opt/app/app.js
        echo "${package_json_b64}" | base64 -d > /opt/app/package.json
        cd /opt/app
        npm install --omit=dev

        cat > /etc/systemd/system/${service_name}.service << 'SERVICE'
        [Unit]
        Description=${service_name} app
        After=network.target

        [Service]
        Environment=PORT=3000
        Environment=AWS_REGION=${region}
        %{ for name, value in environment ~}
        Environment="${name}=${value}"
        %{ endfor ~}
        WorkingDirectory=/opt/app
        ExecStart=/usr/bin/node /opt/app/app.js
        Restart=always
        User=ec2-user

        [Install]
        WantedBy=multi-user.target
        SERVICE

        chown -R ec2-user:ec2-user /opt/app
        systemctl daemon-reload
        systemctl enable ${service_name}.service
        systemctl start ${service_name}.service
```

Then the launch templates and Auto Scaling Groups:

```
locals {
  # What each tier needs to know at boot. Notice the advisor uses the load balancer's DNS name:
  # that reference is the only thing that makes Terraform build the ALB before the advisor's
  # launch template. You never write the order down.
  environment = {
    weather = {
      WEATHER_TABLE  = aws_dynamodb_table.weather.name
      CACHE_SECONDS  = var.cache_seconds
      NWS_USER_AGENT = "(itmo-444-544 weather lab, ${var.contact_email})"
    }
    advisor = {
      WEATHER_URL       = "http://${aws_lb.main.dns_name}:${local.tiers.weather.listener_port}"
      GEMINI_KEY_PARAM  = aws_ssm_parameter.gemini.name
      GEMINI_MODEL      = var.gemini_model
      KEY_CACHE_SECONDS = var.key_cache_seconds
    }
  }

  user_data = {
    for tier, _ in local.tiers : tier => templatefile("${path.module}/user-data.sh.tftpl", {
      service_name     = tier
      region           = var.aws_region
      app_js_b64       = base64encode(file("${path.module}/${tier}-app/app.js"))
      package_json_b64 = base64encode(file("${path.module}/${tier}-app/package.json"))
      environment      = local.environment[tier]
    })
  }
}

resource "aws_launch_template" "tier" {
  for_each = local.tiers

  name                   = "${var.name_prefix}-${each.key}-lt"
  image_id               = data.aws_ami.al2023.id
  instance_type          = var.instance_type
  key_name               = var.key_name
  vpc_security_group_ids = [aws_security_group.app.id]
  user_data              = base64encode(local.user_data[each.key])
  update_default_version = true

  iam_instance_profile {
    name = aws_iam_instance_profile.tier[each.key].name
  }

  tag_specifications {
    resource_type = "instance"
    tags = {
      Name = "${each.key}-app"
    }
  }
}

resource "aws_autoscaling_group" "tier" {
  for_each = local.tiers

  name                      = "${var.name_prefix}-${each.key}-asg"
  min_size                  = 1
  max_size                  = 1
  desired_capacity          = 1
  vpc_zone_identifier       = data.aws_subnets.default.ids
  target_group_arns         = [aws_lb_target_group.tier[each.key].arn]
  health_check_type         = "ELB"
  health_check_grace_period = 120
  force_delete              = true

  launch_template {
    id      = aws_launch_template.tier[each.key].id
    version = aws_launch_template.tier[each.key].latest_version
```

```
  }

  # When the launch template changes (new code, new setting), replace the instance.
  instance_refresh {
    strategy = "Rolling"

    preferences {
      min_healthy_percentage = 0
    }
  }

  tag {
    key                 = "Name"
    value               = "${each.key}-app"
    propagate_at_launch = true
  }
}
```

- `local.environment` is where each tier gets its settings. Look at the advisor's `WEATHER_URL`: it refers to `aws_lb.main.dns_name`. **That one reference is the whole "create the ALB first" logic.** In lab06 you reordered the script and explained why. Here nobody wrote the order, and Terraform worked it out.
- `templatefile` and `base64encode(file(...))` replace the `build_user_data` function and its temporary `user-data-*.sh` files.
- `latest_version` and `instance_refresh` mean that when the user-data changes (new code or a new setting), Terraform publishes a new template version and replaces the instance. In lab06 you published the version and then terminated the instance by hand.
- `force_delete = true` lets `destroy` remove the Auto Scaling Group without waiting for it to scale in, like the `--force-delete` in your delete script.

C9. `outputs.tf`: what to print

```
output "alb_dns_name" {
  description = "The load balancer's address."
  value       = aws_lb.main.dns_name
}

output "weather_url" {
  value = "http://${aws_lb.main.dns_name}:${local.tiers.weather.listener_port}"
}

output "advisor_url" {
  value = "http://${aws_lb.main.dns_name}:${local.tiers.advisor.listener_port}"
}

output "gemini_parameter_name" {
  description = "Where to store your Gemini key."
  value       = aws_ssm_parameter.gemini.name
}

output "weather_table_name" {
  value = aws_dynamodb_table.weather.name
}
```

These replace the `stack_outputs.env` file and the `source stack_outputs.env` lines. The smoke test reads them with `terraform output`.

C10. `terraform.tfvars` and `.gitignore`

Your own values go in `terraform.tfvars`, which Terraform reads automatically. Start from this example:

```
# Copy this file to terraform.tfvars and fill in your own values.
key_name      = "your-lab02-key-pair-name"
contact_email = "you@example.com"
```

```
ssh_cidr      = "203.0.113.7/32"   # your own public address, or "" for no SSH
```

```
        cp terraform.tfvars.example terraform.tfvars
```

Edit it: use your own key pair name (from lab02, without `.pem`), your email address, and your own public address as `x.x.x.x/32` (search the web for "what is my IP"). Set `ssh_cidr = ""` if you do not need SSH.

Then create `.gitignore`, so the state, the plan and your values never reach Git:

```
.terraform/
*.tfstate
*.tfstate.*
tfplan
terraform.tfvars
*.pem
```

✅ **Checkpoint:** `terraform fmt` prints nothing or only file names it fixed, and `terraform validate` (after `init` in Part E) says `Success`.

---

10. Part D: Get a Gemini Key

D1. Create a free key

1. Sign in to Google AI Studio with a **personal** Google account (see Section 4.2).
2. Create an API key from the API keys page. Copy it. It starts with `AIza`.
3. Treat it like a password. Never paste it into a file in your lab folder, a chat, or a screenshot.

D2. Test it once from your computer

Read the key into a shell variable without echoing it. Because you type it at a hidden prompt, it does not land in your shell history.

```
        read -rs -p "Gemini API key: " GEMINI_API_KEY; echo
        export GEMINI_API_KEY

        curl -s "https://generativelanguage.googleapis.com/v1beta/models/gemini-3.1-flash-lite:generateContent" \
          -H "x-goog-api-key: $GEMINI_API_KEY" \
          -H "Content-Type: application/json" \
          -d '{"contents":[{"parts":[{"text":"Suggest what to wear in 45 F weather with light rain, in one
sentence."}]}]}' \
          | python3 -m json.tool | head -30
```

The text of the answer is in `candidates[0].content.parts[0].text`. A `404` usually means the model name needs updating (set `gemini_model` in `terraform.tfvars` to a current Flash-Lite or Flash model), a `400` or `403` means the key was not accepted, and a `429` means you are over the free-tier rate limit. Keep the variable set for Part E.

---

11. Part E: Deploy

E1. Initialize, check and plan

```
        terraform init
        terraform fmt
        terraform validate
        terraform plan -out tfplan
```

- `init` downloads the AWS provider (a minute) and creates `.terraform.lock.hcl`.
- `validate` should print `Success! The configuration is valid.`
- `plan -out tfplan` saves the plan so that `apply` does exactly what you read.

Read the end of the plan. With SSH enabled it says **about 25 to add, 0 to change, 0 to destroy**. Scroll up and find: the load balancer, two target groups, two listeners, two launch templates, two Auto Scaling Groups, two roles and profiles, two policies, one table, one parameter, the security groups and their rules. Everything shows `+`.

E2. Apply

Write down the time (the stack costs about seven cents an hour from now until you destroy it), then apply the saved plan:

```
        date
        terraform apply tfplan
```

There is no `yes` prompt, because you approved the plan by reading it. This takes two to four minutes. When it finishes, Terraform prints the outputs.

E3. Store the real Gemini key

The parameter now exists with the placeholder. Put your key in it, and do it with `--type SecureString` or the parameter would be re-created as plain text:

```
        PARAM=$(terraform output -raw gemini_parameter_name)
        aws ssm put-parameter --name "$PARAM" --type SecureString --value "$GEMINI_API_KEY" --overwrite --region us-e
2
        unset GEMINI_API_KEY
```

Use the region from your `terraform.tfvars` (default `us-east-2`). Until the key is stored, the advisor answers with the fallback, which is the graceful degradation you built in lab06.

E4. Wait for healthy targets and try it

Targets need two to three minutes to report healthy while each instance installs Node.js and your app:

```
        for TG in $(aws elbv2 describe-target-groups --query "TargetGroups[?starts_with(TargetGroupName, 'itmo-444-54
lab07')].TargetGroupArn" --output text --region us-east-2); do
          aws elbv2 describe-target-health --target-group-arn "$TG" \
            --query "TargetHealthDescriptions[].TargetHealth.State" --output text --region us-east-2
        done
```

Wait until both print `healthy`, then:

```
        curl -s "$(terraform output -raw weather_url)/weather?zip=60616" | python3 -m json.tool
        curl -s "$(terraform output -raw weather_url)/weather?zip=60616" | python3 -m json.tool
        curl -s "$(terraform output -raw advisor_url)/advice?zip=60616" | python3 -m json.tool
```

Compare the `cache` field in the first two answers (`miss`, then `hit`). The advice should say `"source": "gemini"` within a minute of storing the key (the app keeps the key for `key_cache_seconds`). Open the `advisor_url` in a browser too.

E5. The smoke test

A smoke test checks that the main paths work. This one asks Terraform for the two addresses, so there is no file to keep in sync.

Create `testing/app_smoke_test.py`:

```
        #!/usr/bin/env python3
```

```
        """Call both tiers through the load balancer and check the answers.

        The addresses come from `terraform output`, so there is no .env file to keep in sync.
        """
        import argparse
        import json
        import subprocess
        import sys
        import time
        import urllib.error
        import urllib.request
        from pathlib import Path


        def terraform_outputs(folder):
            """Return the stack's outputs as a dict, by running `terraform output -json` in the lab folder."""
            result = subprocess.run(["terraform", "output", "-json"], cwd=folder,
                                    capture_output=True, text=True, check=True)
            return {name: item["value"] for name, item in json.loads(result.stdout).items()}


        def get(url, timeout=30):
            """Return (status, parsed JSON or text). HTTP errors are returned, not raised."""
            try:
                with urllib.request.urlopen(url, timeout=timeout) as resp:
                    body, status = resp.read().decode(), resp.status
            except urllib.error.HTTPError as err:
                body, status = err.read().decode(), err.code
            except (urllib.error.URLError, TimeoutError) as err:
                return 0, {"error": f"no connection: {err}"}
            try:
                return status, json.loads(body)
            except ValueError:
                return status, body


        def main():
            parser = argparse.ArgumentParser(description=__doc__)
            parser.add_argument("--zip", default="60616", help="zip code for the happy-path checks")
            parser.add_argument("--weather-url", help="override the weather address (for local testing)")
            parser.add_argument("--advisor-url", help="override the advisor address (for local testing)")
            args = parser.parse_args()

            if args.weather_url and args.advisor_url:
                weather, advisor = args.weather_url, args.advisor_url
            else:
                outputs = terraform_outputs(Path(__file__).resolve().parent.parent)
                weather, advisor = outputs["weather_url"], outputs["advisor_url"]

            results = []

            def check(name, ok, detail=""):
                results.append(ok)
                print(f"{'PASS' if ok else 'FAIL'}  {name}  {detail}")

            status, body = get(f"{weather}/weather?zip={args.zip}")
            check("weather tier returns a forecast", status == 200 and "forecast" in body, f"status {status}")

            status, body = get(f"{weather}/weather?zip={args.zip}")
            check("a repeat call is answered from DynamoDB (cache hit)",
                  status == 200 and body.get("cache") == "hit", f"cache={body.get('cache') if isinstance(body, dict)
body}")

            status, body = get(f"{weather}/history")
            check("history lists the saved lookups", status == 200 and body.get("count", 0) >= 1,
                  f"status {status}, count={body.get('count') if isinstance(body, dict) else '?'}")

            status, body = get(f"{weather}/weather?zip=abc")
            check("weather tier rejects a bad zip with 400", status == 400, f"status {status}")
```

```
    status, body = get(f"{weather}/weather?zip=00000")
    check("weather tier returns 404 for an unknown zip", status == 404, f"status {status}")

    t0 = time.time()
    status, body = get(f"{advisor}/advice?zip={args.zip}")
    ok = status == 200 and bool(body.get("advice"))
    check("advisor tier returns clothing advice", ok, f"status {status}, {time.time() - t0:.1f}s")
    if ok:
        print(f"      source={body.get('source')} model={body.get('model')}")
        print(f"      advice: {body['advice']}")
        check("advice came from Gemini (not the fallback)", body.get("source") == "gemini",
              "fallback means the key is still the placeholder, or the key, quota or model name needs a look")

    status, body = get(f"{advisor}/advice?zip=abc")
    check("advisor tier rejects a bad zip with 400", status == 400, f"status {status}")

    status, body = get(f"{advisor}/")
    check("advisor home page loads", status == 200 and "<form" in str(body), f"status {status}")

    print(f"\n{sum(results)} of {len(results)} checks passed")
    sys.exit(0 if all(results) else 1)


if __name__ == "__main__":
    main()
```

```
python3 testing/app_smoke_test.py
```

All nine checks should pass. If **only** the Gemini check fails, the stack is fine and the key, the model name or the free-tier quota needs attention.

✅ **Checkpoint:** The smoke test passes, and you can say which single line in `compute.tf` made Terraform build the load balancer before the advisor's launch template.

---

12. Part F: Experiments

Run everything from inside `lab07/`. Some steps read an instance's logs over SSH, which needs the `ssh_cidr` setting. This helper finds an instance's public address:

```
export AWS_DEFAULT_REGION=us-east-2
instance_ip() {   # usage: instance_ip weather-app  or  instance_ip advisor-app
  aws ec2 describe-instances \
    --filters "Name=tag:Name,Values=$1" "Name=instance-state-name,Values=running" \
    --query "Reservations[0].Instances[0].PublicIpAddress" --output text
}
```

Step 1. Count the effort

Count what you wrote, then compare. Copy the numbers into your write-up.

```
echo "bash scripts in previous/ (lines, all files):"
cat previous/*.sh | wc -l
echo "bash code lines (no blanks or comments):"
cat previous/*.sh | grep -v '^\s*#' | grep -v '^\s*$' | wc -l
echo "Terraform lines, all files:"
cat *.tf user-data.sh.tftpl | wc -l
echo "Terraform code lines (no blanks or comments):"
cat *.tf user-data.sh.tftpl | grep -v '^\s*#' | grep -v '^\s*$' | wc -l
echo "lines that exist only to clean up (the delete scripts):"
cat previous/delete_*.sh | wc -l
```

Fill in this table (the reference version came out at roughly 420 lines of bash and 430 of Terraform):

	lab06 bash	lab07 Terraform
Lines to **create** the stack		
Lines to **delete** the stack		0
Steps you had to order by hand		0
Helper functions you wrote		0
Retry or wait loops you wrote		0

Terraform is probably **not** shorter for you. Write two sentences on where the effort went down anyway.

Step 2. Prove the order Terraform worked out

You wrote no order, so ask Terraform what it decided:

```
terraform state list
terraform graph | grep -E "aws_lb\.main|launch_template" | head -12
```

Find the arrow that connects the advisor's launch template to `aws_lb.main`. Write one sentence on what would happen if you removed the `WEATHER_URL` line from `compute.tf`: would the order change, and would the advisor still work?

Step 3. Run apply again

```
terraform plan
```

It should say `No changes. Your infrastructure matches the configuration.` Write one sentence comparing this with running `create_weather_stack.sh` twice in lab06. (Hint: the bash script printed "already exists, skipping" lines. What did it check, and what did it not check?)

Step 4. Change one setting and watch a rolling replacement

Edit `terraform.tfvars` and add:

```
cache_seconds = 60
```

```
terraform plan
```

Read the plan carefully. It should show **only the weather tier** changing: its launch template (a new version, because the user-data changed) and its Auto Scaling Group (`~` an update in place that starts an instance refresh). The advisor, the load balancer, the table and the roles are **not** touched, because nothing they depend on changed. Apply it, and watch the replacement:

```
terraform apply
aws ec2 describe-instances --filters "Name=tag:Name,Values=weather-app" "Name=instance-state-
name,Values=pending,running,shutting-down" \
  --query "Reservations[].Instances[].[InstanceId,State.Name]" --output text
aws autoscaling describe-instance-refreshes --auto-scaling-group-name itmo-444-544-lab07-weather-asg \
  --query "InstanceRefreshes[0].[Status,PercentageComplete]" --output text
```

When the new instance is healthy, check that it picked up the change:

```
ssh -i <your-key>.pem ec2-user@"$(instance_ip weather-app)" "systemctl cat weather | grep CACHE"
```

You should see `CACHE_SECONDS=60`. Write down how many commands this took, and how many it took in lab06 (edit `.env`, run the create script, find the instance, terminate it). Then set `cache_seconds` back by deleting the line and applying again.

Step 5. Cause drift and watch Terraform notice

Drift is a difference between your code and reality. Make some by hand. Change the weather target group's health check path:

```
        TG=$(aws elbv2 describe-target-groups --names itmo-444-544-lab07-weather-tg --query
"TargetGroups[0].TargetGroupArn" --output text)
        aws elbv2 modify-target-group --target-group-arn "$TG" --health-check-path /nope
        terraform plan
```

Terraform shows a `~` that sets `path` back to `/`. Do **not** apply yet. The weather targets will turn unhealthy within about 30 seconds, because `/nope` returns `404`. Check:

```
        aws elbv2 describe-target-health --target-group-arn "$TG" --query "TargetHealthDescriptions[].TargetHealth.St
--output text
```

Now apply, and watch the targets recover:

```
        terraform apply
```

Now delete something entirely. Remove the advisor's listener and see what the advisor page does:

```
        LB=$(terraform output -raw alb_dns_name)
        LISTENER=$(aws elbv2 describe-listeners --load-balancer-arn "$(aws elbv2 describe-load-balancers --names itmo
444-544-lab07-alb --query 'LoadBalancers[0].LoadBalancerArn' --output text)" --query "Listeners[?
Port==\`8081\`].ListenerArn" --output text)
        aws elbv2 delete-listener --listener-arn "$LISTENER"
        curl -m 5 -s -o /dev/null -w "advisor page: %{http_code}\n" "http://$LB:8081/" || echo "advisor page: no answ
        terraform plan | tail -5
        terraform apply -auto-approve
```

The plan should say `1 to add`. Write down the difference from lab06's `ensure_target_group`, which found the group already existing and said "skipping" **without checking that the health check was still right**.

Step 6. Who can do what? (least privilege, proved)

IAM can tell you what a role may do without trying it:

```
        TABLE_ARN=$(aws dynamodb describe-table --table-name itmo-444-544-lab07-weather-lookups --query Table.TableAr
output text)
        PARAM_ARN=$(aws ssm get-parameter --name "$(terraform output -raw gemini_parameter_name)" --query Parameter.A
-output text)
        check() {   # role, action, resource
          ROLE_ARN=$(aws iam get-role --role-name "$1" --query Role.Arn --output text)
          printf "%-40s %-22s " "$1" "$2"
          aws iam simulate-principal-policy --policy-source-arn "$ROLE_ARN" \
            --action-names "$2" --resource-arns "$3" \
            --query "EvaluationResults[0].EvalDecision" --output text
        }
        check itmo-444-544-lab07-weather-role dynamodb:PutItem "$TABLE_ARN"
        check itmo-444-544-lab07-weather-role ssm:GetParameter "$PARAM_ARN"
        check itmo-444-544-lab07-advisor-role ssm:GetParameter "$PARAM_ARN"
        check itmo-444-544-lab07-advisor-role dynamodb:GetItem "$TABLE_ARN"
```

The first and third lines should say `allowed`, and the second and fourth `implicitDeny`. Write one sentence on what an attacker who takes over the **advisor** tier gets, and what they do not get.

Step 7. The key never touched Terraform

You stored the real key with the CLI. Check that Terraform never saw it:

```
terraform plan | tail -3
grep -c "AIza" terraform.tfstate
grep -n "not-set-yet" terraform.tfstate | head -2
```

The plan says `No changes`, even though the stored value is different from `not-set-yet`. That is `ignore_changes` working. The `grep -c` should print `0`, because your key is nowhere in the state. The placeholder **is** in the state, and it is harmless. Now remove `ignore_changes` in your head: write two sentences on what the next `terraform apply` would do to your key, and what would then be in the state.

Step 8. Break the key and watch the fallback

Overwrite the key with a wrong value (the advisor still works, with the fallback), and then restore it:

```
PARAM=$(terraform output -raw gemini_parameter_name)
aws ssm put-parameter --name "$PARAM" --type SecureString --value "not-a-real-key" --overwrite
sleep 70
for i in 1 2; do
  curl -s "$(terraform output -raw advisor_url)/advice?zip=60616" | python3 -c "import sys,json;
d=json.load(sys.stdin); print(d['source'], '-', d['advice'])"
done
ssh -i <your-key>.pem ec2-user@"$(instance_ip advisor-app)" "sudo journalctl -u advisor --no-pager | grep
gemini_error | tail -2"
```

Every answer says `fallback`, yet nothing returned an error. Then restore your real key with the same command and hidden prompt as Part E3. Write down one problem with a design where users never see an error and the only evidence is a log line that nobody reads.

Step 9. The Lambda exercise (no resources, written answer)

Section 3 said the advisor could be an AWS Lambda function behind the same load balancer. Without building it, answer from your own files:

1. List every resource in `compute.tf`, `iam.tf`, `network.tf` and `alb.tf` that exists **only because the advisor runs on an EC2 instance**.
2. List what would **stay** (the parameter, the policy, the listener, the load balancer, and so on).
3. List what would be **new** (a function resource, a permission that lets the load balancer call it, and the work of changing the code to handle a Lambda event).
4. For each of your three lists, say whether Terraform or Lambda removed the effort. What did you gain, and what new problem did you take on (think: cold starts, time limits, how you would test it)?

Fill in the table below as your answer. It is the main written result of this lab.

Advisor on EC2 (this lab)	Advisor on Lambda	Effort moved to
Launch template and user-data		
Auto Scaling Group and instance refresh		
App security group rule for port 3000		
Instance profile and role		
Node.js install and systemd service		
Health check and grace period		

Step 10. Destroy, and prove nothing is left

Write down the time, then destroy everything:

```
date
terraform destroy
```

Read the plan (it should say about **25 to destroy**, and nothing to add), type `yes`, and note how long it takes. This one command replaced your 76 line delete script, its retry loop for the security group, and the separate delete script for the tables. Work out how many hours the stack ran, and multiply by the hourly cost in Section 4.1. That is your estimate for the lab.

Now prove nothing is left. The `Project` tag you set once in `versions.tf` makes this easy:

```
terraform state list
aws resourcegroupstaggingapi get-resources --tag-filters Key=Project,Values=itmo-444-544-lab07 \
  --query "ResourceTagMappingList[].ResourceARN" --output text
aws elbv2 describe-load-balancers --query "LoadBalancers[].LoadBalancerName"
aws autoscaling describe-auto-scaling-groups --query "AutoScalingGroups[].AutoScalingGroupName"
aws ec2 describe-instances --filters "Name=instance-state-name,Values=pending,running,stopping,stopped" \
  --query "Reservations[].Instances[].[InstanceId,Tags[?Key=='Name'].Value|[0]]" --output text
aws dynamodb list-tables
aws ssm describe-parameters --query "Parameters[].Name"
aws iam list-roles --query "Roles[?starts_with(RoleName, 'itmo-444-544')].RoleName"
```

Everything should be empty. The tagging command can briefly list a resource that is still finishing deletion, so run it again after a minute. Finally, delete the key in Google AI Studio too, since the parameter was only a copy, and check **Billing and Cost Management** in a few hours.

---

13. Review Questions and Troubleshooting

13.1. Review questions

Answer these in your write-up.

Topic	What this lab did	The question to answer
Declarative and imperative	Terraform compares files, state and AWS, then decides the steps	Give one thing the imperative bash version could do that is hard in Terraform, and one thing Terraform did that the bash version never did
State file	`terraform.tfstate` records every resource and ID	What would happen if you lost it? What would happen if two people applied from different copies? How do teams solve this?
Secrets in state	The Gemini parameter uses `ignore_changes` and the key is stored with the CLI	Where else could a secret reach the state or a plan file? Why is a plan file also sensitive?
Plan before apply	`plan -out` then `apply tfplan`	What does reading a plan protect you from? Which plan symbol worries you most, and why?
Version pinning	`~> 5.0` and the lock file	What could go wrong with no version pin, and why do we commit `.terraform.lock.hcl`?
`for_each` or `count`	Both tiers come from one map, and SSH uses `count`	Why does `for_each` handle removing one tier better than `count` would?
Ownership	Terraform owns a new security group instead of editing lab02's	What goes wrong when a tool and a person both edit the same resource?
Drift	You changed a health check by hand	How could a team find drift automatically and regularly? Who should be allowed to change things by hand?
Blast radius	Changing `cache_seconds` touched only the weather tier	What decides how much one change touches, and how could you shrink it further?
Least privilege	Each role names one resource, and the policy refers to the resource's own ARN	Why is a reference safer than a typed-in ARN?
Terraform and Lambda	Section 3 and Step 9	In your own words: what does Terraform have in common with Lambda, and what is the one difference that matters most for how you would use each?
Destroy	One command removes everything	Which resources would you protect from a careless `destroy` in a real environment, and how (look up `prevent_destroy`)?

13.2. Troubleshooting

Symptom	Likely cause	Fix
`terraform: command not found`	Not installed, or not on your `PATH`	Install it (Part 0.2) and open a new terminal
`No valid credential sources found`	Terraform cannot find your AWS credentials	Run `aws sts get-caller-identity`. If that fails, fix your CLI credentials first
`terraform init` cannot download the provider	No internet access, or a proxy blocks the registry	Check your network. On a school network ask for access to `registry.terraform.io`
`Error: Unsupported argument` or `Unsupported block type`	A typo, or the provider version differs from the lab	Check spelling against the file in this document, and run `terraform version`
`Invalid value for variable` on `ssh_cidr`	Not a CIDR such as `203.0.113.7/32`	Add the `/32`
`No value for required variable`	`terraform.tfvars` is missing or not in the folder	Copy the example file and fill it in
`InvalidKeyPair.NotFound`	`key_name` does not match a key pair in this region	Use the name without `.pem`, and check `aws ec2 describe-key-pairs`
`DuplicateLoadBalancerName`, `DuplicateTargetGroupName` or `EntityAlreadyExists`	A resource with that name is left from an earlier run	Destroy the earlier run, or delete the leftover by hand, or change `name_prefix`
`ParameterAlreadyExists`	The parameter from an earlier run is still there	Delete it with `aws ssm delete-parameter --name ...`, then apply again
`Error acquiring the state lock`	Another `terraform` command is still running, or one was interrupted	Wait, or if you are sure none is running, `terraform force-unlock <ID>`
Targets stay `unhealthy`	The app crashed at boot, or the first boot is still installing	Wait 3 minutes. Then SSH in and run `sudo journalctl -u weather -n 30 --no-pager` (or `advisor`)
Instances cannot install Node.js or reach APIs	The outbound security group rule is missing	Check that `app_out` exists in `network.tf`
Advice is always `fallback`	The key is still the placeholder, the key was rejected, or the model name changed	`sudo journalctl -u advisor --no-pager | grep gemini_error` on the advisor instance. Store the key (Part E3) and wait for `key_cache_seconds`
Gemini returns 404	The model name does not exist	Set `gemini_model` in `terraform.tfvars` and apply
`/history` returns 503 or cache is `unavailable`	The weather role cannot reach the table	`sudo journalctl -u weather --no-pager | grep db_error`. Check `WEATHER_TABLE` with `systemctl cat weather`
`terraform plan` wants to change something you did not touch	Drift, or a provider update changed a default	Read the `~` lines. If it is drift, apply to fix it. If it is a provider change, decide whether to accept it
`DependencyViolation` deleting a security group	The load balancer's network interfaces are still releasing	Wait. The provider retries for several minutes. If it times out, run `terraform destroy` again
`destroy` finishes but a resource remains	It was created outside Terraform, or was removed from the state	Find it with the tag command in Step 10 and delete it by hand
`ssh` times out	`ssh_cidr` is not your current address	Update `terraform.tfvars` and apply. Terraform changes only the one rule
User-data too large	Added code pushed it over 16 KB	Keep the apps small. Each tier is about 10 to 12 KB with its code embedded

14. Submission Checklist

Submit these 10 items:

- [ ] Output of `terraform version` and `aws sts get-caller-identity` (hide the account number).
- [ ] Part A: the `plan` before the first apply, the second `apply` showing `No changes`, the in-place update plan (`~`) and the replacement plan (`-/+`), with a sentence on why `-/+` is the one to be careful about.
- [ ] Output of `terraform fmt`, `terraform validate` and the last lines of `terraform plan` showing the number of resources to add.
- [ ] Output of `app_smoke_test.py` with all nine checks passing, and a screenshot of the advisor page.
- [ ] Steps 1 to 3: your effort table with line counts, the `terraform graph` output with your sentence, and the `No changes` plan with your sentence comparing it to running the bash script twice.
- [ ] Steps 4 and 5: the plan for `cache_seconds = 60`, the instance refresh status and new `CACHE_SECONDS=60` line, and the drift plans with your sentence about `ensure_target_group`.
- [ ] Steps 6 and 7: the IAM simulator results with your sentence about the advisor tier, and the `No changes` plan, the `grep -c` count of `0` and your sentences about `ignore_changes`.
- [ ] Steps 8 and 9: the fallback answers with the `gemini_error` journal line and your sentence about silent fallback, and the completed Lambda table with your answer to question 4.
- [ ] Short answers: how Terraform is like AWS Lambda and where the comparison stops, what Terraform did **not** make easier, and your answers to the Section 13.1 review questions.
- [ ] Step 10: the `terraform destroy` plan and total time, your cost estimate (hours multiplied by the hourly cost), and the verification commands showing nothing remains.

---

15. Key Terms Glossary

- **Infrastructure as Code (IaC):** Describing infrastructure in files that can be reviewed, versioned and re-run, instead of clicking in a console.
- **Imperative:** Code that lists the steps to take, in order. Your bash and boto3 scripts.
- **Declarative:** Code that describes the end state you want, and leaves the steps to a tool. Terraform.
- **Terraform:** A tool that reads declarative files and makes AWS (or another cloud) match them.
- **OpenTofu:** An open source fork of Terraform with the same commands and files.
- **HCL:** HashiCorp Configuration Language, the syntax of Terraform files.
- **Provider:** The plug-in that knows how to talk to one service's API. The AWS provider turns resource blocks into AWS API calls.
- **Resource:** A block for a thing Terraform creates and manages.
- **Data source:** A block that looks up something that already exists.
- **Variable:** An input to the configuration. **Output:** A value printed after apply. **Local:** A named value inside the configuration.
- **Reference and dependency:** Using one resource's attribute in another. Terraform builds its order from these.
- **Dependency graph:** The map of what depends on what, from which Terraform decides the order and what can run at the same time.
- **`for_each` and `count`:** Ways to create several copies of a resource from one block.
- **Plan:** A preview of exactly what an apply would create, change or destroy.
- **Apply and destroy:** Carrying out a plan, and removing everything in the state.
- **State:** Terraform's record of what it built and the real IDs. It can hold secrets, so protect it.
- **Remote state and locking:** Keeping the state in a shared place that only one person can change at a time.
- **Lock file (`.terraform.lock.hcl`):** The record of the exact provider version, committed so everyone gets the same one.
- **Drift:** A difference between your code and the real infrastructure, usually caused by a change made by hand.
- **Idempotent:** Running it again changes nothing. A second `apply` with no edits does nothing.
- **Instance refresh:** An Auto Scaling feature that replaces instances when the launch template changes.
- **`lifecycle` and `ignore_changes`:** Settings that change how Terraform treats a resource. `ignore_changes` tells it to leave an attribute alone.
- **Blast radius:** How much one change can affect.
- **AWS Lambda:** A service that runs your function code when an event arrives, without you managing servers, and bills per invocation.
- **Cold start:** The extra delay when Lambda has to prepare a new environment for a function.

- **Least privilege:** Giving an identity only the permissions it needs, on only the resources it needs.
- **Policy simulator:** The IAM feature that reports whether a role would be allowed to make a call, without making it.
- **Instance profile:** The wrapper that attaches an IAM role to an EC2 instance.
- **SecureString:** A Parameter Store type that is encrypted at rest and needs permission to read.
- **Graceful degradation and fallback:** Giving a reduced but useful answer when a dependency fails, and labelling it.
- **Time to live (TTL):** A DynamoDB setting that deletes items after a timestamp passes.

---

16. References

- Terraform documentation: https://developer.hashicorp.com/terraform/docs
- Install Terraform: https://developer.hashicorp.com/terraform/install
- Terraform language: resources, data sources, `for_each` and `lifecycle`: https://developer.hashicorp.com/terraform/language
- The `templatefile` function: https://developer.hashicorp.com/terraform/language/functions/templatefile
- Terraform state and sensitive data: https://developer.hashicorp.com/terraform/language/state/sensitive-data
- Terraform AWS provider documentation: https://registry.terraform.io/providers/hashicorp/aws/latest/docs
- OpenTofu: https://opentofu.org
- AWS Lambda developer guide: https://docs.aws.amazon.com/lambda/latest/dg/welcome.html
- Application Load Balancer targets that are Lambda functions: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/lambda-functions.html
- Auto Scaling instance refresh: https://docs.aws.amazon.com/autoscaling/ec2/userguide/asg-instance-refresh.html
- National Weather Service API documentation: https://www.weather.gov/documentation/services-web-api
- Zippopotam.us zip code API: https://api.zippopotam.us
- Google AI Studio and the Gemini API documentation (models, rate limits and data use terms): https://ai.google.dev/gemini-api/docs
- AWS Systems Manager Parameter Store: https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html
- IAM policy simulator: https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_testing-policies.html